

StockPilot

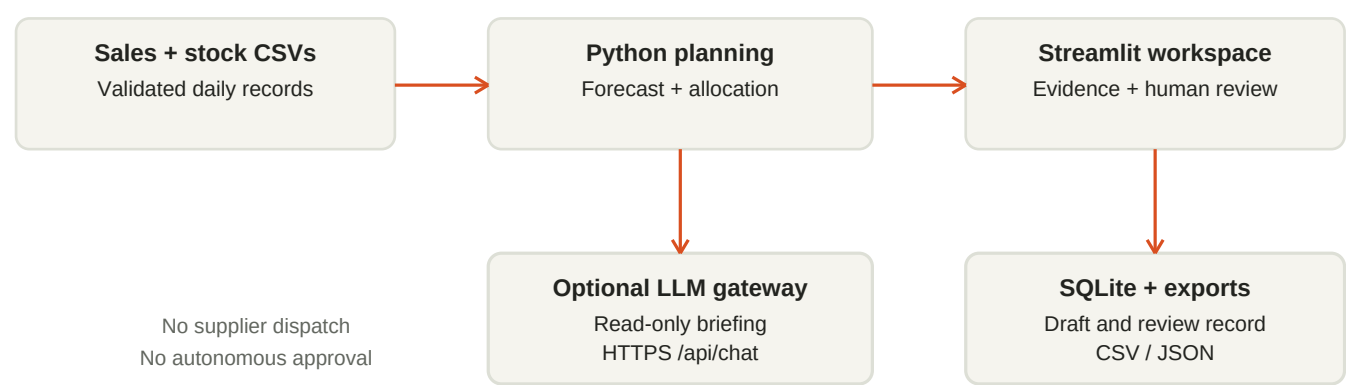
Purchasing intelligence, explained.

Team Skibidi Slicers | NUS-ISS Show Me Your Agent Hackathon
Chia Chong En | Mohd Arif Fiqry Bin Mohd Suffian
Tan Kai Hao Gerald | Trevis Tan

25 September 2026 | Repository: github.com/xensica/nus-iss

01 / SYSTEM OVERVIEW

StockPilot prepares explainable purchase recommendations across a small retail catalogue. The current application is **NUS-ISS/purchase-order-agent/purchase_demo/** in the repository. It supports up to 15 imported products, a 12-product sample and three simulated suppliers. Calculations and state changes are deterministic application logic; optional AI explains existing evidence.



Module(s), relative to purchase_demo/	Responsibility
demo_app.py / visuals.py	Six-screen Streamlit interface, charts, review controls and escaped HTML components.
data_io.py / uci_import.py	CSV validation, catalogue assembly, sample data and transaction preparation.
engine.py / catalogue.py	Forecasting, supplier allocation, catalogue budget totals and stress replay.
gateway.py	Bounded, read-only AI briefing over the organiser gateway.
storage.py / migrations.py	SQLite draft deduplication and guarded simulated review.
evaluation.py / tests/	Held-out forecast evaluation, purchasing replay and automated checks.

Scope: Control room, Sourcing desk, Disruption lab, Order review, Data studio and Evidence. Root purchasing scripts and single_product_reference.py are earlier prototypes, not the current entry point.

02 / DATA CONTRACTS

Inputs, provenance and validation

File / required columns	Optional fields and defaults
sales.csv date, product_id, units	Daily totals; date must be YYYY-MM-DD. One row per product/date.
inventory.csv product_id, current_stock	incoming_units: 0; incoming_day: 1 when no incoming stock; name: product ID; category: Imported; unit_price_cents: 200.

Daily CSV validation

UTF-8 (BOM accepted); maximum 10 MB for uploaded byte inputs and 100,000 rows per file. Column names must be unique and required columns present. Product IDs use 1-40 ASCII letters, digits, hyphens or underscores. Units and stock are integers from 0 to 1,000,000. Reference prices are positive integer SGD cents up to 100,000. Incoming stock requires a day from 1 to 14. Inventory permits at most 15 products; both files must contain identical product IDs.

Atomic loading and date semantics

Both files are validated and assembled before replacing the active dataset. Duplicate daily rows or inventory products are rejected. Missing daily observations stay unknown; they are not silently converted to zero. Inventory has no date column: the operator must align it with the selected analysis date. After a daily upload, the UI initially sets that date to the latest sales date; verify it before relying on a recommendation.

```

sales.csv
date,product_id,units
2026-09-23,C0F,34

inventory.csv
product_id,current_stock,incoming_units,incoming_day,unit_price_cents
C0F,136,0,1,200

```

Schema illustration only: this single sales row is not sufficient for a reliable forecast or the evaluation holdout.

Dataset representation

An assembled dataset contains products (including sorted history), original sales/inventory rows, a source label and a 16-character SHA-256-derived fingerprint. The fingerprint identifies the serialized inputs; it is not a signed authenticity guarantee. Session state holds uploaded data and analysis. Persisted order payloads retain calculation evidence and settings.

UCI-format transaction preparation

The optional importer reads InvoiceNo, StockCode, Description, Quantity, InvoiceDate and UnitPrice from a CSV under 100 MB. It excludes cancellations, nonpositive quantities/prices, invalid dates/codes, missing values and quantities above an explicit cap (default 1,000). It never reads CustomerID. Returns are excluded rather than netted.

The UI requires ledger-completeness confirmation before absent transaction days are filled with zero. Up to 12 products are selected using training-period trading-day coverage, with at least 42 calendar days. Stock is simulated at five days of recent mean and purchase reference price fixed at S\$2.00. GBP sale prices are not used as SGD purchase costs. The original UCI workbook has not been validated in this run.

03 / DECISION ENGINE

Forecast, quantity and supplier allocation

Forecast calculation

For analysis date t , retain observations on or before t within the previous 28 calendar days. For each of the next 14 days, average matching weekdays. If fewer than two matching observations exist, use the recent overall mean and flag review. No recent observations blocks the forecast. Annual seasonality and learned event effects are outside scope.

```
baseline[d] = ceil(mean(recent matching-weekday units))
demand[d]   = ceil(baseline[d] * (1 + adjustment/100)
                * (1 + event_uplift[d]/100))
safety      = ceil(mean(demand[1:14]) * buffer_days)
quantity     = max(0, sum(demand[1:14]) + safety - stock - incoming)
reorder_pt  = sum(demand[1:7]) + safety
```

The ranges above are inclusive mathematical day ranges. Event uplift applies only within the configured event window and outside Normal trading. Percentage arithmetic currently uses floating point; a known rounding issue is disclosed on page 7.

Simulated supplier	Price factor / fee	Arrival / MOQ / capacity	Quality / on-time score
A - Express	1.0x / S\$15	Day 2 / 50 / 1,200	95 / 98
B - Economy	0.8x / S\$10	Day 7 / 100 / 2,000	85 / 90
C - Standard	0.9x / S\$12	Day 4 / 50 / 1,200	90 / 95

Price factors multiply each product's reference purchase price, rounded to integer cents. Fees apply per product/supplier allocation. Scores and delivery days are assumptions, not calibrated probabilities or guarantees.

Exact, bounded search

Filter suppliers by quality, reliability and approved status. Enumerate single-supplier plans and all integer splits across each pair, for an exact requested quantity up to 2,000 units per product. Every allocation must meet MOQ/capacity. Incoming stock and supplier shipments arrive at the start of their delivery day; every end-of-day balance must be nonnegative, and final stock must meet the safety buffer. Optional urgent units must arrive by the explicit urgent deadline.

Objective, budget and statuses

Minimize units x price plus fees; break cost ties using fewer shipments. Return ready, no_purchase_needed, over_budget or blocked. A zero quantity still checks timing and safety feasibility. Compute savings only when a feasible single-supplier comparator exists. Catalogue totals show budget pressure, and drafting independently enforces the selected batch's shared budget. No global catalogue allocation optimization is performed.

Stress replay and search limits

Stress tests replay the same allocation with raised demand and delayed new deliveries; existing incoming stock keeps its date. The search does not optimize three-supplier splits, extra MOQ stock, cross-product freight or shared supplier capacity. With three suppliers and a 14-day horizon, enumeration is bounded by $O(S^2 \times Q \times H)$ per product.

04 / AGENT & RECORD INTEGRITY

Read-only AI, human-controlled review

Optional AI briefing contract

gateway.brief() loads LLM_GATEWAY_URL, LLM_GATEWAY_API_KEY and LLM_MODEL from the server-side purchase_demo/.env file. The URL must use HTTPS. Each call posts to /api/chat with X-API-Key, model, messages, stream=false and num_predict=550; timeout is 45 seconds per call, with at most four calls. No model training or fine-tuning is performed.

Step	Required model action	Application-supplied evidence
1	read_inventory	Source label, product count and dataset fingerprint.
2	read_recommendations	Per-SKU quantity, shortage day, status, cost and allocations.
3	check_budget	Total cost, budget and over-budget flag.
4	answer	A text briefing using the supplied evidence.

Tool requests must exactly match {"type": "tool", "name": "NAME"}, with no arguments or extra keys. The final object must contain only type="answer" and a string message. Unexpected order, malformed responses or transport errors produce a generic error; calculated recommendations remain available. Returned text has the configured key redacted. Live gateway compatibility is not verified.

Draft payload and persistence

storage.draft_payload() accepts selected products with ready plans and actual order lines, then checks their combined cost against the shared budget. Version-2 payloads store source, dataset fingerprint, settings, product forecasts/overrides and order lines. save_draft() hashes sorted JSON with SHA-256; an INSERT OR IGNORE on a unique fingerprint deduplicates identical payloads. Batch IDs use SP- plus eight UUID hexadecimal characters.

```
SQLite: purchase_demo/demo_orders.db
batches(id PRIMARY KEY, fingerprint UNIQUE, status, payload,
        reviewed_at, reviewed_role)

DRAFT -- Manager review --> APPROVED_SIMULATION
DRAFT -- Manager review --> REJECTED
```

Review integrity

The UI requires a confirmation checkbox for approval. storage.review() requires the simulated Manager role and updates only a DRAFT row whose stored payload exactly equals the reviewed string. A changed or already-reviewed record is not updated. Review time is UTC. Startup migration adds reviewed_role when absent and preserves legacy rows. Order CSV and audit JSON downloads are available.

Security boundary

Roles are a selector, not authenticated identity. The model has no write, approval or dispatch capability. SQL uses bound parameters; custom HTML escapes uploaded labels; CSV export prefixes formula-like strings. Gateway errors hide raw transport/model content. A shared deployment still needs authentication and data isolation: all sessions using the same SQLite file can access its review records.

05 / RUN & HOST

Reproduce locally, then verify the host

Commands below start the current application from a fresh repository checkout. Windows execution was tested locally; the Ubuntu steps are a deployment recipe, not evidence that an AWS instance is already serving the application.

Windows / PowerShell

Install Python and Git, then run the following from a chosen working directory. The included launcher binds to 127.0.0.1:8504 and applies the application theme.

```
git clone https://github.com/xensica/nus-iss.git
cd nus-iss/NUS-ISS/purchase-order-agent
py -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r purchase_demo/requirements.txt
.\purchase_demo\Start-StockPilot.ps1
```

Ubuntu / Lightsail preview

Proposed instance: Singapore, Ubuntu 24.04 LTS, general purpose, dual-stack and 2 GB RAM. Attach a static IP. Install the following dependencies and start a loopback-only preview. Access it through an SSH tunnel, or put a configured HTTPS reverse proxy in front of it for external access. [R2, R3]

```
sudo apt-get update
sudo apt-get install -y python3-venv python3-pip git
git clone https://github.com/xensica/nus-iss.git
cd nus-iss/NUS-ISS/purchase-order-agent
python3 -m venv .venv
.venv/bin/python -m pip install -r purchase_demo/requirements.txt
.venv/bin/python -m streamlit run purchase_demo/demo_app.py \
  --server.address 127.0.0.1 --server.port 8504 \
  --server.headless true --browser.gatherUsageStats false
```

From your own terminal, an SSH tunnel can map local port 8504 to server port 8504 using your Lightsail key and instance IP. The loopback preview is not a public deployment URL. Streamlit address/port options follow its official configuration reference. [R1]

Gateway configuration

Copy .env.example to .env beside demo_app.py and set the organiser-provided HTTPS gateway URL, private key and supported model identifier. Keep .env out of Git. The calculation workflow works without these variables. The model identifier in .env.example is a configuration example, not confirmation of current availability.

Before public demonstration

Configure HTTPS, WebSocket-capable proxying, authenticated access and a startup service; keep port 8504 private behind the proxy. Restrict SSH access and check both IPv4 and IPv6 firewall rules. Back up SQLite consistently and confirm restart persistence. Record the deployed commit, reachable URL and screenshots for the separate deployment-evidence submission. [R1-R3]

Dependencies: streamlit >=1.63,<2; pandas >=2,<4; requests >=2.32,<3; python-dotenv >=1,<2; pytest >=8,<10. Altair is imported via the installed Streamlit dependency set. Version ranges are not a frozen deployment lockfile.

06 / VERIFICATION & EVALUATION

Measured on 25 September 2026

Automated checks: 31 passed in 7.39 seconds. The successful rerun used Windows and Python 3.13.3. The initial sandbox attempt hit temporary-folder permissions; the rerun completed outside that restriction. Tests use isolated databases and do not dispatch orders.

```
# From purchase-order-agent (Windows)
.\.venv\Scripts\python.exe -m pytest purchase_demo/tests -q -p no:cacheprovider
.\.venv\Scripts\python.exe -m purchase_demo.evaluation
```

On Linux, use .venv/bin/python in place of the Windows interpreter path.

Coverage includes CSV rejection and round trips, sparse/stale data, future-data exclusion, event windows, quantity/timing checks, split allocations, budget blocking, stress replay, migrations, deduplication, role/payload guards, evaluation reproducibility, mocked gateway sequence/errors, all six UI screens and a simulated approval flow. Component tests check escaping and shipment order; they are not a browser screenshot audit.

Holdout design

Source: deterministic synthetic 12-product sample, 84 daily records each; fingerprint **389855f934b519cc**. Training ends 9 September 2026; holdout is 10-23 September (14 days). No products were skipped. Evaluation requires at least 42 consecutive observations. The last-week and 28-day moving-average baselines use the same holdout.

Forecast method	MAE (units)	MAPE (%)	WAPE (%)
Weekday average	1.68	6.64	6.69
Last week	2.11	8.49	8.39
Moving average	2.70	10.87	10.71

MAE = mean absolute error; MAPE averages absolute percentage errors over nonzero actuals; WAPE = total absolute error / total actual units x 100. Metrics above pool all product-day observations. WAPE is undefined when all actual units are zero.

Purchasing replay	Cost (SGD)	Stockout product-days	Lost units
Agent	4,467.35	0	0
Reorder-point baseline	4,127.68	20	306

Both policies start with five days of training-mean stock, no incoming deliveries and one order decision. The baseline selects the cheapest threshold-eligible supplier without enforcing daily stock coverage. Lost sales are not backlogged. Totals sum across products: stockout product-days are not unique calendar days.

Interpretation: The agent costs S\$339.67 more in this replay and avoids the baseline's simulated shortages. This is a cost/service trade-off, not a purchasing-savings result. Synthetic results do not establish real-store accuracy, preparation time, seven-day warning performance or commercial benefit.

Installed versions in this run: Streamlit 1.63.0; pandas 3.0.5; requests 2.34.2; python-dotenv 1.2.3; pytest 9.1.1. Passing tests do not resolve the calculation issues on page 7.

07 / LIMITATIONS & REFERENCES

What is verified, and what remains open

Known finding	Effect and next check
Percentage rounding	Floating-point arithmetic can make 25 units + 12% round up to 29 instead of 28. Replace with an appropriate exact/decimal calculation and add boundary cases.
Urgent supplier comparison	compare_suppliers() does not forward optional urgent parameters into its single-supplier check. Align its comparison with the actual plan constraints.
Zero-quantity blocker text	An infeasible urgent constraint with zero net quantity can incorrectly blame late incoming stock. Distinguish shortage timing from urgent-constraint conflicts.
Simulated approvals / shared records	Manager roles are unauthenticated. Approvals do not reserve stock or create confirmed arrivals. Reconcile overlapping batches and add identity/isolation before operational use.
Bounded optimization	One or two suppliers, exact quantity, at most 2,000 units per product. No cross-product capacity or freight consolidation. A blocked result applies to this restricted search.
External verification	Live gateway connectivity, AWS hosting and the original UCI workbook remain unverified. This document supplies no public deployment evidence.

Reproduction and release trace

The local application Python modules matched the clean repository checkout at commit **fa252df9cbe600e09161bfe90b3e95164eb6f71d**. Tests and evaluation in this document ran against the local purchase_demo copy. Reproduce evaluation with `python -m purchase_demo.evaluation`; add `--data purchase_demo/sample_data` to use the bundled CSV files. Before release, capture the actual deployed revision and dependency versions.

REFERENCES & SOURCE MAP

Project source: [StockPilot application at the reviewed commit](#). Architecture and algorithms: engine.py, catalogue.py, gateway.py. Inputs: data_io.py, uci_import.py. Persistence: storage.py, migrations.py. Verification: evaluation.py and tests/. Current scope and audit disclosures: README.md.

[R1] Streamlit: [Configuration reference](#) - server address/port and protection settings.

[R2] AWS: [Lightsail firewall and port mappings](#) - public access controls.

[R3] AWS: [Create and attach a static IP](#) - stable instance addressing.

Documentation checked 25 September 2026. Deployment steps describe proposed configuration, not completed deployment.

UCI-format support references the source schema recorded in the repository; no original dataset download or independent dataset validation was performed for this document. No claim of AWS certification or direct Amazon Bedrock integration is made: the AI path is the organiser's HTTP gateway.